

# Unifying Streaming Data Reorganization and Interactive Real-Time Observability in Large Clusters

by  
Ankush Jain

A DISSERTATION PROPOSAL  
Submitted in partial fulfillment of  
the requirements for the degree of

DOCTOR OF PHILOSOPHY  
Electrical and Computer Engineering  
Carnegie Mellon University

Committee:  
Prof. George Amvrosiadis (Chair)  
Prof. Greg Ganger  
Prof. Andy Pavlo  
Prof. Vyas Sekar  
Dr. Brad Settlemyer

CARNEGIE MELLON UNIVERSITY  
Pittsburgh, Pennsylvania  
July 2025

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Contents</b>                                                   | <b>i</b>  |
| <b>1 Introduction</b>                                             | <b>2</b>  |
| 1.1 Motivation: Efficiency Bottlenecks in BSP . . . . .           | 2         |
| 1.2 Our Work: A Comprehensive Study of BSP Efficiency . . . . .   | 3         |
| 1.3 Thesis Statement . . . . .                                    | 4         |
| 1.4 Document Structure . . . . .                                  | 5         |
| <b>2 Background and Landscape</b>                                 | <b>6</b>  |
| 2.1 The BSP Paradigm and Synchronized Cluster Computing . . . . . | 6         |
| 2.2 Anatomy of Modern Clusters . . . . .                          | 6         |
| 2.3 Data Movement and I/O Challenges . . . . .                    | 6         |
| 2.4 In-Situ Analytics: Community Drivers and Status Quo . . . . . | 6         |
| 2.5 Scientific Code Ecosystem . . . . .                           | 6         |
| <b>3 CARP — Adaptive In-Situ Range Partitioning</b>               | <b>8</b>  |
| 3.1 Relevant Background and Architectural Rationale . . . . .     | 8         |
| 3.2 Motivation and Context . . . . .                              | 8         |
| 3.3 Approach . . . . .                                            | 8         |
| 3.4 Key Results . . . . .                                         | 8         |
| 3.5 Broader Impact . . . . .                                      | 8         |
| <b>4 AMR — Diagnosing and Optimizing Adaptive Mesh Codes</b>      | <b>9</b>  |
| 4.1 Background and Motivation . . . . .                           | 9         |
| 4.2 Load-Balancing Policies and Baselines . . . . .               | 9         |
| 4.3 The Diagnostic Journey . . . . .                              | 9         |
| 4.4 Evolution of the Telemetry Pipeline . . . . .                 | 9         |
| 4.5 Key Results and Policy Development . . . . .                  | 9         |
| 4.6 Broader Lessons . . . . .                                     | 10        |
| <b>5 ORCA — Generalizing Observability and Control</b>            | <b>11</b> |
| 5.1 Background and Motivation . . . . .                           | 11        |
| 5.2 Approach and Architecture . . . . .                           | 11        |
| 5.3 Key Results and Capabilities . . . . .                        | 11        |
| 5.4 Remaining Work and Timeline . . . . .                         | 11        |
| 5.5 Broader Impact . . . . .                                      | 11        |
| <b>References</b>                                                 | <b>12</b> |

## Abstract

The Bulk Synchronous Parallel (BSP) paradigm powers our most critical computational tools, from large-scale scientific simulations to distributed machine learning. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery. Yet the tightly synchronized nature of BSP creates two distinct challenges: on the data path, bursts of checkpointed state strain storage and analytics infrastructure, while on the compute path, global synchronization amplifies sensitivity to execution variability. While traditionally addressed by separate tools, this thesis demonstrates that both problems can be unified by treating them as challenges of organizing and processing streaming data for insight.

To address the data path, we present CARP, an adaptive range partitioner that reorders data in-situ. To enable efficient range queries on continuous physical attributes (e.g., particle energy) without expensive post-processing sorts, CARP adaptively tracks highly skewed and evolving data distributions as they are generated. By doing so, it approximates the query performance of a perfect sort with no ingestion overhead, achieving up to 5x faster end-to-end performance than prior work and query latencies up to 100x faster than other in-situ techniques.

We then turn to the compute path, where our investigation revealed that genuine load imbalance in adaptive codes was frequently confounded by deeper sources of variability—hardware faults, system noise, and unpredictable cross-stack interactions—exacerbated by global synchronization. Isolating these effects required fine-grained telemetry that standard tools could not provide, compelling us to develop custom, query-driven analytics pipelines. This diagnostic process yielded CPLX, a tunable placement policy that improves runtime by up to 21.6%. More importantly, it exposed the critical need for queryable, real-time feedback to rapidly characterize performance bottlenecks.

Our experiences with streaming data reorganization in CARP and the diagnostic challenges encountered during our load-balancing work culminated in ORCA, a framework for online, declarative, and dynamic observability. ORCA reconstructs on-demand views of distributed system state, streaming only the telemetry required to synthesize the requested views. These views can evolve over time through a control path that supports consistent, asynchronous updates. ORCA uses columnar telemetry formats, distributed query planning, and operator push-down to minimize data movement and support low-latency introspection. By providing a single, programmable substrate for query-driven dataflows, ORCA bridges long-standing silos between performance diagnostics and in-situ data analytics, offering a unified response to the data and compute challenges of large-scale parallel systems.

# 1 Introduction

## 1.1 Motivation: Efficiency Bottlenecks in BSP

The Bulk Synchronous Parallel (BSP) paradigm powers some of our most critical computational tools, from large-scale scientific simulations in astrophysics and climate science to distributed training of foundation models in machine learning. These workloads—consisting of large parallel computations interspersed with I/O and global synchronization—currently operate at the scale of tens of thousands of accelerator nodes [1–4] and continue to grow with time [5]. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery.

The tightly synchronized nature of BSP creates two unique challenges that strain modern computing infrastructure. On the data path, bursts of checkpointed state create massive I/O demands on the storage and analytics infrastructure [6]. On the compute path, frequent global synchronization amplifies the system’s sensitivity to execution variability [7–9]. Performance bottlenecks that would be manageable in loosely coupled systems, such as minor OS-level interruptions or thermal throttling, become critical scaling barriers when every worker must wait for the slowest participant.

We now describe the status quo and outstanding efficiency challenges along the data and compute paths.

**Data Management.** The explosive growth in scientific data volumes has made in-situ analytics an essential strategy for reducing I/O overhead and accelerating time-to-insight. While widely used for visualization and occasionally adopted for data transformation or staging, current in-situ systems are fragmented, limited, and purpose-built. Most frameworks hardcode specific operations or assume fixed data layouts, with no support for composing multiple indexing strategies at ingest time. Existing systems like FastQuery build auxiliary indices such as bitmaps, while others like DeltaFS perform high-throughput hash partitioning suitable for point queries—but neither supports efficient range queries. No general-purpose mechanism exists for building query-efficient layouts dynamically and incrementally during ingestion. As a result, range query performance still hinges on costly post-processing sorts, undermining the core premise of in-situ data processing.

**Compute Efficiency.** The parallel inefficiency of BSP codes is widely ascribed to load imbalance, but there has been little systematic effort to characterize its sources or validate mitigation strategies. The standard approach—offline trace analysis—is both cumbersome and disconnected from real-time system context. Traces are large, opaque, and typically analyzed long after execution, often by researchers without access to the hardware environment that produced them. Profiling tools, while lightweight, expose only coarse-grained summaries and miss transient effects that may drive synchronization delays. As a result, performance tuning remains ad-hoc and reactive, lacking the infrastructure for targeted, context-aware introspection during execution.

## 1.2 Our Work: A Comprehensive Study of BSP Efficiency

Our work developed systems and algorithms to advance workflow efficiency of parallel codes on both the compute and data paths. These systems demonstrated that significant efficiency gains were possible and revealed key insights for making these wins accessible on production clusters. This experience led us to develop ORCA, a general-purpose approach that enables efficiency wins in production environments. We describe each system in turn, highlighting the contributions and insights that collectively motivated our unified approach.

### 1.2.1 Data Path: Streaming Data Reorganization

**Motivation.** Our work was motivated by the need for write-efficient ingestion techniques that for scientific data analyses. Analyses on continuous physical properties such as wavefronts or energy bands require efficient range queries. All existing solutions for efficient range queries require reordering of on-disk data via postprocessing. Not only would an in-situ ingestion system deliver efficiency gains, it would also complete the toolkit of fundamental in-situ data organization primitives.

**Challenges.** Our goal was to develop a purely streaming system that would never reorder data once written to disk, without making any assumptions about data distribution characteristics. Our characterization study found scientific data distributions to be highly skewed and evolving over time. Any static range partitioning scheme would become imbalanced, creating write bottlenecks and query hotspots. Our system needed to ingest data with unknown, dynamic, and evolving data distributions, maintain full utilization of the storage system, and scale to thousands of cores with minimal application runtime overhead.

**Our Contribution.** CARP (Continuous Adaptive Range Partitioner), an adaptive partitioner that adapts to arbitrary key distributions and partitions them in a single streaming pass. CARP provides query latencies comparable to those of a full sort without any ingestion overhead, making it 5× faster than prior work.

### 1.2.2 Compute Path: Understanding and Mitigating Load Imbalance

**Motivation.** Poor utilization in adaptive mesh codes a well-known problem. We wanted to create load-balancing mechanisms that would incorporate fine-grained runtime telemetry from these codes, and incorporate them into a placement policy that conferred meaningful runtime gains.

**Challenges.** Our efforts to collect and interpret telemetry, and create models of code performance that could be exploited in a placement policy encountered several persistent challenges.

1. Raw telemetry measurements did not correlate with expected behavior, forcing deep investigation of cross-stack performance anomalies, and revealing issues with hardware, network stack parameters, operation ordering etc. These constituted runtime noise that had to be eliminated before telemetry and placement effects could be reliably observed.

2. Identifying, isolating, and tracing transient performance issues in a large parallel code required collecting and analyzing a large volume of telemetry. Existing performance tools were unhelpful: profiling tools did not provide the required granularity, while traces were slow and not analyzable.
3. Need efficient placement computation on NP-hard problem within tight timing constraints.

**Our Contribution.** A placement policy (CPLX) with tunable control over two placement trade-offs: compute load balance and communication locality. By balancing these two objectives, CPLX provides upto 21.6% runtime improvement over optimized baselines.

### 1.2.3 ORCA: Transferring Lessons Learned to General-Purpose Observability

**Motivation.** While our previous efforts were successful in demonstrating the potential for efficiency gains on both data and compute paths, realizing these gains on production clusters remained elusive. Realizing data path efficiency requires a general-purpose data management system that can use the best combination of indexes for a given analytics workload. Realizing compute path efficiency is even more challenging: our work was conducted on an in-house research cluster with no operational, security, or vendor constraints. Realizing these benefits in production clusters requires efficient workflows to trace variability to its root cause, which may or may not be amenable to algorithmic solutions.

**Our Approach.** We are developing a general-purpose dataflow framework for cluster observability using SQL semantics and Arrow. ORCA (Observability with Realtime Control and Aggregation) uses a tree-based overlay network with distributed query planning and a async control path.

**Challenges.** No existing template exists for cluster-scale streaming analytics. The system requires complex distributed coordination with multiple interacting components while meeting strict performance requirements to avoid impacting application critical paths.

## 1.3 Thesis Statement

This thesis asserts that overcoming the primary data and compute bottlenecks in large-scale BSP applications requires a paradigm shift from specialized, siloed tools to a unified, online *sense-making* engine, and demonstrates that the architectural foundation for this shift is a framework that treats both in-situ analytics and performance diagnostics as a single problem of composing query-driven dataflows.

### 1.3.1 Potential Impact

**Immediate Impact.** Application-level observability becomes a first-class capability. ORCA replaces brittle traces and low-resolution profilers with a lightweight, always-on, query-driven system that supports real-time diagnosis and runtime tuning. Developers can rapidly inspect distributed

behavior, toggle probes, and react to anomalies without modifying application code or halting execution.

**Longer-Term Impact.** ORCA evolves into a flexible substrate for in-situ analytics and adaptive control. Its open architecture supports programmable analysis pipelines, integration of diverse telemetry sources, and runtime composition of custom indexing and aggregation strategies. Over time, it can subsume visualization, introspection, and control workflows into a unified framework that prioritizes insight per unit of data moved, enabling principled trade-offs between data fidelity, performance, and resource usage.

## 1.4 Document Structure

The remainder of this document is organized as follows.

## 2 Background and Landscape

### 2.1 The BSP Paradigm and Synchronized Cluster Computing

- What is BSP? Why is it foundational for both HPC and large-scale ML?
- Timestepped execution, collectives, and synchronization barriers.
- Impact on application design: sensitivity to stragglers and the importance of uniform progress.

### 2.2 Anatomy of Modern Clusters

- Cluster resources: nodes, memory hierarchies, accelerators (e.g., GPUs).
- **Networking:**
  - HPC: specialized high-speed fabrics (e.g., InfiniBand, Aries) for collectives, plus Ethernet control networks.
  - ML clusters: backend networks for data, frontend for orchestration.
- Storage infrastructure: parallel filesystems (Lustre, GPFS, BeeGFS), burst buffers, SSD tiers.

### 2.3 Data Movement and I/O Challenges

- Bursty checkpointing and collective I/O phases in scientific apps.
- The network/storage bandwidth gap as a persistent architectural problem.
- Checkpointing and trace collection as stressors for both storage and system performance.
- The difference between data written for analysis (HPC) and for recoverability (ML).

### 2.4 In-Situ Analytics: Community Drivers and Status Quo

- Traditional “persist-then-analyze” model and its breakdown at scale.
- Why in-situ emerged: reducing time-to-insight, bandwidth needs, and storage costs.
- Overview of major in-situ paradigms:
  - Tightly coupled: libraries embedded in simulation.
  - Loosely coupled: co-processing with staging/burst nodes.
  - Overlay networks: aggregation and scalable monitoring.
  - In-situ indexing: hash-based (DeltaFS), range (missing).
- Limitations of each and the context for new approaches.

### 2.5 Scientific Code Ecosystem

- Landscape of open-source and institutional codes: how science gets done at scale.
- **VPIC:** what it is, scientific significance, I/O demands.
- **AMR codes:** (e.g., Parthenon, FLASH, Enzo) for multiscale, adaptive modeling.
- Other representative codes and domains (brief nod to climate, molecular dynamics, etc.).

- Common challenges: data volumes, run duration, complexity, and the resulting analysis needs.

## 3 CARP — Adaptive In-Situ Range Partitioning

### 3.1 Relevant Background and Architectural Rationale

- Prior in-situ approaches were either tightly coupled to the application (hard to adapt and extend) or focused on fast, unordered ingestion (hash partitioning), sacrificing query efficiency.
- Range queries are critical in scientific analysis, but traditional solutions required expensive post-processing sorts—architecturally at odds with the streaming, in-memory nature of modern HPC I/O.
- The network-to-storage bandwidth gap and the bursty, append-heavy I/O patterns of checkpointing make streaming, adaptive, one-pass reorganization architecturally optimal.

### 3.2 Motivation and Context

- Adaptive range partitioning was the last missing primitive for general in-situ data organization.
- Scientific datasets are highly skewed and dynamic, demanding solutions that can adjust in real time.

### 3.3 Approach

- CARP adaptively reorganizes data during application I/O, discovering and adjusting range partitions on the fly.
- Operates fully in-situ, without extra passes or reordering after the fact.

### 3.4 Key Results

- Delivers near-sorted query performance for range queries, with zero ingestion overhead.
- Outperforms prior work: up to 5x faster than post-processing and up to 100x lower query latency than in-situ indexing baselines.
- Robust across extremes of skew and evolving data distributions.

### 3.5 Broader Impact

- Completes the set of architectural primitives for in-situ data organization.
- Introduces the “state-driven adaptive control loop” pattern, a theme carried forward in the thesis.

## 4 AMR — Diagnosing and Optimizing Adaptive Mesh Codes

### 4.1 Background and Motivation

- Block-structured AMR (Adaptive Mesh Refinement) codes are widely used for large-scale simulations in physics and engineering, but are notoriously difficult to optimize at scale.
- Synchronization and frequent mesh refinement create straggler effects; performance is traditionally diagnosed as a load-balancing problem.

### 4.2 Load-Balancing Policies and Baselines

- Common practice: work placement policies (e.g., Space-Filling Curves, naive block distribution) assume uniform costs and static assignment.
- Prior literature and production codes rely on heuristics; empirical tuning is difficult, and “obvious” improvements often fail at scale.

### 4.3 The Diagnostic Journey

- Initial attempts to optimize placement exposed more complex bottlenecks: hardware faults, system noise, and non-obvious sources of variability.
- Standard profiling tools were insufficient—transient effects and cross-stack anomalies went undetected.

### 4.4 Evolution of the Telemetry Pipeline

- Necessity drove a bottom-up diagnostic process: custom code instrumentation, direct CSV logging, and iterative data analysis.
- The ad-hoc telemetry stack evolved into a query-driven, structured workflow, borrowing concepts from modern OLAP and observability.
- Adoption of programmable tools (eBPF, OLAP engines) enabled the root-cause diagnosis that classical HPC tools could not deliver.

### 4.5 Key Results and Policy Development

- The CPLX placement policy: a tunable hybrid balancing load and communication locality, outperforming prior baselines by up to 21.6%.
- Main finding: true bottlenecks often lie outside what classical load balancing or placement can solve—highlighting the need for deeper, programmable, telemetry-driven diagnosis.

## 4.6 Broader Lessons

- Performance optimization for modern AMR codes is fundamentally empirical and diagnostic, not purely algorithmic.
- Motivates the need for a general, programmable approach to cluster telemetry and control—setting the stage for ORCA.

## 5 ORCA — Generalizing Observability and Control

### 5.1 Background and Motivation

- Lessons from CARP and AMR revealed the need for a general, programmable framework for both analytics and real-time control in large clusters.
- Existing tools provided either narrow observability, limited programmability, or ad-hoc steering—none delivered unified, declarative, scalable control.

### 5.2 Approach and Architecture

- ORCA introduces a decoupled overlay network: application, aggregators, and controller communicate via high-performance RPC.
- Built on distributed query planning (Apache DataFusion) for SQL-driven, in-memory analytics on streaming telemetry.
- Real-time control via a Timestep-Linked Two-Phase Commit (TS2PC) protocol ensures consistent, synchronized updates across all ranks.

### 5.3 Key Results and Capabilities

- Demonstrates scalable, low-overhead analytics at line-rate bandwidths, enabling “analyze-then-persist” at cluster scale.
- Makes dynamic, cluster-wide control practical—enabling online tuning, interactive workflows, and safe application reconfiguration in situ.
- Provides a declarative, extensible substrate for both scientific introspection and systems-level observability.

### 5.4 Remaining Work and Timeline

- TODO

### 5.5 Broader Impact

- ORCA subsumes fragmented observability and steering solutions under a unified, data-centric paradigm.
- Establishes the general, programmable control loop as a foundational primitive for future HPC and ML systems.

## References

- [1] Sambit Das et al. “Large-Scale Materials Modeling at Quantum Accuracy: Ab Initio Simulations of Quasicrystals and Interacting Extended Defects in Metallic Alloys”. In: SC ’23: International Conference for High Performance Computing, Networking, Storage and Analysis. 2023.
- [2] Ryan Stocks et al. “Breaking the Million-Electron and 1 EFLOP/s Barriers: Biomolecular-Scale Ab Initio Molecular Dynamics Using MP2 Potentials”. In: SC ’24: International Conference for High Performance Computing, Networking, Storage and Analysis. 2024.
- [3] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs]. Pre-published.
- [4] *Meta Trains Llama 4 on a 100,000+ H100 GPU Supercluster - NADDOD Blog*.
- [5] Robi Rahman. *AI Training Cluster Sizes Increased by More than 20x since 2016*. Epoch AI. 2024.
- [6] Assaf Eisenman et al. “{Check-N-Run}: A Checkpointing System for Training Deep Learning Recommendation Models”. In: 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 2022.
- [7] Torsten Hoefer, Timo Schneider, and Andrew Lumsdaine. “Characterizing the Influence of System Noise on Large-Scale Applications by Simulation”. In: SC ’10: The 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis. 2010.
- [8] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. “The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q”. In: SC ’03: International Conference for High Performance Computing, Networking, Storage and Analysis. 2003.
- [9] Orcun Yildiz et al. “On the Root Causes of Cross-Application I/O Interference in HPC Storage Systems”. In: 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS). 2016.